Order Management Platform – Enterprise Architecture

Oct 29, 2026 · @Ayyappa kammila

1. Project overview

The platform takes a customer order from any channel and carries it through payment, stock, shipping, returns and SAP billing, using 30 Spring Boot microservices that talk mostly through Kafka.

Client type. A large US omnichannel retailer selling general merchandise (electronics, home, apparel, essentials) through a website, a React Native app and physical stores. Customers can choose home delivery, ship-from-store or buy-online-pick-up-in-store (BOPIS).

Business problem. Orders were handled by an old monolith that could not scale on sale days, sent invoices to SAP in nightly batches, and gave support agents no single view of an order. The new platform must:

- Accept up to about 3,000 orders per minute on peak sale days without losing or duplicating any order.
- Charge customers exactly once, and refund exactly once.
- Reserve stock across warehouses and stores in real time.
- Hand orders to logistics partners and track them to delivery.
- Post sales orders and invoices to SAP SD close to real time, not overnight.
- Give support agents one portal to search, fix and retry orders.

What the platform does, end to end.

1. Customer places an order (web, app or store) through a BFF and the API Gateway.
2. Order is validated, priced and taxed, then saved with an outbox event.
3. A saga reserves stock, authorizes payment and runs fraud checks.
4. Fulfillment picks the best ship-from location; shipping books a carrier and gets a label.
5. On shipment, payment is captured and an invoice is created.
6. Camel routes post the order (ORDERS05) and invoice (INVOIC02) IDocs to SAP.
7. Returns and refunds flow back through the same services, with compensation if any step fails.
8. Every event lands in Elasticsearch for fast search, and in the admin portal for agents.

Learning-project note. This is a reference design for a personal build. You do not need to run all 30 services at once. Section 15 suggests a build order.

Tech stack at a glance

Layer	Choice
Language / framework	Java 17, Spring Boot 3.x, Spring Cloud 2023.x
Front end	Angular 17 (admin portal)
Messaging	Apache Kafka (Amazon MSK in prod), ActiveMQ Artemis (JMS)
Integration	Apache Camel 4 (SAP, REST, SFTP, JMS components)
Databases	PostgreSQL, MongoDB, Elasticsearch, Redis
Security	Keycloak, OAuth 2.0, JWT
Resilience	Resilience4j
AI	Spring AI (Amazon Bedrock model, PGVector)
DevOps	Jenkins, Docker, Kubernetes (EKS), Helm, Argo CD, Terraform
Observability	Actuator, Micrometer, Prometheus, Grafana, ELK, Jaeger

2. The 30 microservices

14 services are core (heavy business logic); 16 are supporting (edge, data, platform, helpers). Every service is a separate Spring Boot app with its own database and its own Git repo.

2.1 Service catalog

#	Service	Type	Responsible for	Talks to
1	api-gateway	Supporting	Single entry point, routing, JWT check, rate limiting	All BFFs, Keycloak, Eureka
2	discovery-service	Supporting	Eureka service registry	All services register here

#	Service	Type	Responsible for	Talks to
3	config-server	Supporting	Central config from a Git repo, per environment	All services read at startup
4	mobile-bff	Supporting	API shaped for the React Native app	order, cart, catalog-sync, customer, order-query
5	web-bff	Supporting	API shaped for the storefront	Same as mobile-bff
6	admin-bff	Supporting	API for the Angular admin portal	order-query, order, payment, returns, integration services, chatbot
7	cart-service	Core	Cart, checkout validation, place-order request	pricing, inventory, order
8	order-service	Core	Order create, status, cancel; owns the order record; writes outbox	Kafka, cart, pricing
9	order-orchestrator	Core	Saga: runs reserve stock, fraud, payment, fulfill; sends compensations	Kafka (commands and replies)
10	pricing-service	Core	Price, promotions, coupons, order totals	catalog-sync, tax-service, Redis
11	tax-service	Supporting	Sales tax by US state/ZIP via external tax API	External tax provider
12	inventory-service	Core	Stock per location, reservations, release, ATP	Kafka, store-service
13	payment-service	Core	Authorize, capture, void, refund; transaction history; reconciliation	payment-gateway-adapter, Kafka
14	payment-gateway-adapter	Core	Camel routes to Stripe and PayPal, webhooks	Stripe, PayPal, payment-service
15	fraud-service	Core	Rules for risky orders and flagged refunds	Kafka, customer-service

#	Service	Type	Responsible for	Talks to
16	fulfillment-service	Core	Picks ship-from location, splits orders, BOPIS	inventory, store-service, shipping
17	shipping-service	Core	Rates, labels, tracking status	integration-logistics
18	integration-logistics	Core	Camel routes to carriers (FedEx, UPS style APIs), JMS, SFTP	Carriers, ActiveMQ, shipping
19	returns-service	Core	Return requests (RMA), eligibility, return labels	order, shipping, payment, fraud
20	invoice-service	Core	Builds invoices on shipment, invoice numbers	Kafka, order, pricing
21	integration-sap	Core	Camel routes: orders and invoices to SAP SD as IDocs, status back	SAP, Kafka, Redis
22	customer-service	Supporting	Customer profile, addresses, preferences	Keycloak (user id)
23	catalog-sync-service	Supporting	Consumes PIM product events, stores product snapshots	PIM (Kafka), MongoDB
24	store-service	Supporting	Store master data, hours, BOPIS pickup slots	inventory, fulfillment
25	order-query-service	Supporting	CQRS read side: order search and history	Kafka, Elasticsearch
26	notification-service	Supporting	Email, SMS, push on order events	Kafka, email/SMS providers
27	audit-service	Supporting	Stores who-did-what for agent actions and money events	Kafka, S3
28	file-exchange-service	Supporting	Camel SFTP jobs: settlement files, carrier manifests, SAP batch files	SFTP servers, S3, payment
29	reporting-service	Supporting	Daily sales, refunds, SLA reports for business	Read replicas, order-query

#	Service	Type	Responsible for	Talks to
30	chatbot-service	Supporting	Spring AI support chatbot for agents	order-query, shipping, payment, PGVector

Keycloak, Kafka, Debezium (Kafka Connect) and ActiveMQ are infrastructure, not counted in the 30.

2.2 Main REST APIs

All paths are behind the gateway and versioned as `/api/v1/...`. Internal-only APIs use `/internal/v1/...` and are not routed by the gateway.

Service	Method	Endpoint	Purpose
mobile-bff / web-bff	GET	/api/v1/{channel}/home	Home data: recent orders, cart count
mobile-bff / web-bff	GET	/api/v1/{channel}/orders/{orderId}	Order details with items, payment, tracking in one call
admin-bff	GET	/api/v1/admin/orders/{orderId}/timeline	Full order timeline for agents
cart-service	POST	/api/v1/carts	Create cart
cart-service	POST	/api/v1/carts/{cartId}/items	Add item
cart-service	PUT	/api/v1/carts/{cartId}/items/{itemId}	Change quantity
cart-service	POST	/api/v1/carts/{cartId}/checkout	Validate and place order
order-service	POST	/internal/v1/orders	Create order from checkout
order-service	GET	/api/v1/orders/{orderId}	Order by id

Service	Method	Endpoint	Purpose
order-service	POST	/api/v1/orders/{orderId}/cancel	Cancel order
order-service	PATCH	/internal/v1/orders/{orderId}/status	Update status (used by saga)
order-orchestrator	GET	/internal/v1/sagas/{sagaId}	Saga state, for support
pricing-service	POST	/internal/v1/prices/quote	Price a cart with promos and tax
pricing-service	POST	/api/v1/coupons/validate	Check a coupon code
tax-service	POST	/internal/v1/tax/calculate	Tax for lines and address
inventory-service	GET	/api/v1/inventory/{sku}/availability	Available-to-promise by ZIP
inventory-service	POST	/internal/v1/reservations	Reserve stock
inventory-service	DELETE	/internal/v1/reservations/{reservationId}	Release stock
payment-service	POST	/internal/v1/payments/authorize	Authorize amount with gateway token
payment-service	POST	/internal/v1/payments/{paymentId}/capture	Capture on shipment
payment-service	POST	/api/v1/payments/{paymentId}/refunds	Refund (agent or auto)
payment-service	GET	/api/v1/payments/order/{orderId}	Payment history for an order

Service	Method	Endpoint	Purpose
payment-gateway-adapter	POST	/webhooks/stripe	Stripe webhook (signature checked)
payment-gateway-adapter	POST	/webhooks/paypal	PayPal webhook (signature checked)
fraud-service	POST	/internal/v1/fraud/check	Score an order or refund
fraud-service	GET	/api/v1/fraud/cases	Flagged cases for review
fulfillment-service	POST	/internal/v1/fulfillments	Create fulfillment plan for an order
fulfillment-service	GET	/api/v1/fulfillments/order/{orderId}	Shipments for an order
shipping-service	POST	/internal/v1/shipments/rates	Carrier rates
shipping-service	POST	/internal/v1/shipments/{id}/label	Book carrier, get label
shipping-service	GET	/api/v1/shipments/{id}/tracking	Tracking events
integration-logistics	POST	/webhooks/carriers/{carrier}	Carrier tracking callback
returns-service	POST	/api/v1/returns	Create return request
returns-service	GET	/api/v1/returns/{returnId}	Return status

Service	Method	Endpoint	Purpose
returns-service	POST	/api/v1/returns/{returnId}/approve	Agent approves flagged return
invoice-service	GET	/api/v1/invoices/order/{orderId}	Invoices for an order
integration-sap	GET	/api/v1/integrations/sap/messages?status=FAILED	Failed IDocs
integration-sap	POST	/api/v1/integrations/sap/messages/{id}/retry	Retry from DLQ
integration-logistics	POST	/api/v1/integrations/logistics/messages/{id}/retry	Retry from DLQ
customer-service	GET	/api/v1/customers/me	Logged-in customer profile
customer-service	PUT	/api/v1/customers/me/addresses/{id}	Update address
catalog-sync-service	GET	/internal/v1/products/{sku}	Product snapshot
store-service	GET	/api/v1/stores?zip=	Nearby stores
store-service	GET	/api/v1/stores/{storeId}/pickup-slots	BOPIS slots
order-query-service	GET	/api/v1/orders/search	Search by id, email, phone, date, status (cursor paging)
order-query-service	GET	/api/v1/customers/me/orders	Customer order history (search_after cursor)

Service	Method	Endpoint	Purpose
notification-service	GET	/api/v1/notifications/order/{orderId}	What was sent to the customer
audit-service	GET	/api/v1/audit?entity=ORDER&id=	Audit trail
file-exchange-service	POST	/api/v1/files/jobs/{jobName}/run	Run an SFTP job now
reporting-service	GET	/api/v1/reports/daily-sales?date=	Daily sales report
chatbot-service	POST	/api/v1/chat	Ask the support chatbot
All services	GET	/actuator/health, /actuator/prometheus	Health and metrics

3. Main business flows

Every flow that changes money or stock runs as a saga over Kafka; only reads and quick validations are synchronous REST. Topic names follow `oms.<domain>.<event>`.

3.1 Order placement

1. App calls `POST /api/v1/mobile/checkout` on mobile-bff through api-gateway (JWT checked).
2. mobile-bff calls cart-service `POST /carts/{id}/checkout`.
3. cart-service calls pricing-service (REST) for final totals; pricing calls tax-service and reads product prices from Redis or catalog-sync.
4. cart-service calls inventory-service `GET availability` as a quick pre-check.
5. cart-service calls order-service `POST /internal/v1/orders` with an `Idempotency-Key` header.
6. order-service, in ONE database transaction, inserts `orders`, `order_lines` and an `outbox_event` row (OrderCreated). Status = PENDING.
7. Debezium reads the outbox table and publishes `oms.order.created`.
8. order-service returns 201 with orderId; the customer sees "Order placed".

9. order-orchestrator consumes `oms.order.created` and starts the saga (next flows).
10. order-query-service and notification-service also consume it (search index, confirmation email).

3.2 Saga: stock, fraud, payment

1. Orchestrator sends `oms.inventory.reserve-cmd` → inventory-service reserves stock → replies `oms.inventory.reserved` (or `reserve-failed`).
2. Orchestrator sends `oms.fraud.check-cmd` → fraud-service → `fraud.approved` / `fraud.review` / `fraud.rejected`.
3. Orchestrator sends `oms.payment.authorize-cmd` → payment-service → payment-gateway-adapter (Camel) calls Stripe or PayPal with the card token and idempotency key → `payment.authorized` / `payment.failed`.
4. All OK → orchestrator tells order-service to set status CONFIRMED, and sends `oms.fulfillment.create-cmd`.
5. Any failure → compensation in reverse: void authorization, release reservation, set order CANCELLED, notify customer.

3.3 Inventory update

1. On `reserve-cmd`, inventory-service locks the row (`SELECT ... FOR UPDATE`) and moves quantity from `on_hand` to `reserved`.
2. On shipment (`oms.shipment.shipped`), reserved quantity is decremented for good.
3. On cancel or saga failure, the reservation is released.
4. Warehouse and store stock feeds arrive every 15 minutes as SFTP files (file-exchange-service) and as Kafka events from store POS; inventory-service applies them as absolute snapshots.
5. Every change publishes `oms.inventory.changed`, which updates availability caches.

3.4 Shipping and fulfillment

1. fulfillment-service consumes `fulfillment.create-cmd` and picks the ship-from node: nearest DC or store with stock, splitting the order if needed.
2. For BOPIS, it books a pickup slot in store-service and skips the carrier.
3. For delivery, it calls shipping-service for rates, then `label`.
4. shipping-service publishes `oms.shipping.label-requested`; integration-logistics (Camel) calls the carrier API, gets tracking number and label PDF (stored in S3).
5. Warehouse management system confirms pick-pack-ship over JMS (ActiveMQ); integration-logistics turns that into `oms.shipment.shipped`.

6. Carrier tracking callbacks hit `/webhooks/carriers/{carrier}` and become `oms.shipment.tracking-updated`, up to DELIVERED.
7. On `shipment.shipped`, payment-service captures the amount for those lines, and invoice-service creates the invoice.

3.5 Returns and refunds

1. Customer calls `POST /api/v1/returns` from the app (via BFF).
2. returns-service checks eligibility (return window, item returnable) and calls fraud-service.
3. Clean case → `return.approved` automatically; flagged case (high value, damaged claim, repeated returns) → `return.review` queue in admin portal.
4. Approved → shipping-service generates a return label (or in-store return).
5. When the warehouse or store receives the item (JMS or POS event) → `oms.return.received`.
6. payment-service consumes it and calls `refunds` with idempotency key `refund-{returnId}`; gateway result comes back by webhook.
7. `oms.payment.refunded` updates order status, inventory (restock if sellable), notification, and triggers a credit-memo IDoc to SAP.
8. Cancel before shipping = no return needed: void the authorization instead of refunding.

3.6 SAP invoice flow (full)

1. Order CONFIRMED → integration-sap consumes `oms.order.confirmed`.
2. Redis check `SETNX sap:order:{orderId}` (TTL 7 days); key exists → skip (duplicate).
3. Camel maps the order to an **ORDERS05** IDoc (header E1EDK01, partners E1EDKA1, items E1EDP01) and sends it with the Camel SAP component (`sap-idoc-destination`).
4. SAP creates the sales order; SAP returns a status IDoc (ALEAUD) through a Camel `sap-idoc-server` route → `oms.sap.order-posted` with the SAP order number, saved on the order.
5. Shipment confirmed → invoice-service creates invoice → `oms.invoice.created`.
6. integration-sap builds an **INVOIC02** IDoc (E1EDK01 header, E1EDP01 lines, E1EDS01 totals), referencing the SAP sales order, and sends it the same way.
7. SAP posts billing; status 53 = success → `oms.sap.invoice-posted`. Status 51 = error → message parked as FAILED.
8. Timeouts retry 3 times with exponential backoff (2s, 4s, 8s); then the message goes to `oms.sap.dlq` and an alert fires. Data errors go to DLQ at once, no retry.

9. Agents see failed IDocs in the admin portal and click Retry after the data is fixed.
10. Refunds follow the same route as a credit memo (INVOIC02 with credit document type).
11. Nightly, file-exchange-service drops a summary file on SFTP for SAP finance reconciliation.

4. Database design

Each service owns its own database; no service reads another service's tables. PostgreSQL holds money and orders, MongoDB holds flexible documents, Elasticsearch serves search, Redis holds short-lived keys.

4.1 Database per service

Service	Database	Why
order-service	PostgreSQL `order_db`	ACID writes + outbox in one transaction
order-orchestrator	PostgreSQL `saga_db`	Saga state must survive restarts
cart-service	Redis (active carts) + PostgreSQL `cart_db` (saved carts)	Fast, short-lived
pricing-service	PostgreSQL `pricing_db` + Redis cache	Promotions are relational rules
inventory-service	PostgreSQL `inventory_db`	Row locks for reservations
payment-service	PostgreSQL `payment_db`	Money, audit history
payment-gateway-adapter	Redis (idempotency)	Stateless otherwise
fraud-service	PostgreSQL `fraud_db`	Rules + cases
fulfillment-service	PostgreSQL `fulfillment_db`	Plans and shipments
shipping-service	PostgreSQL `shipping_db`	Shipments, tracking
returns-service	PostgreSQL `returns_db`	Return lifecycle
invoice-service	PostgreSQL `invoice_db`	Invoice numbers must be unique, gap-free

Service	Database	Why
integration-sap	PostgreSQL `integration_db` + Redis	Message log, retries, idempotency
integration-logistics	PostgreSQL `integration_db` (own schema) + MongoDB	Carrier payloads vary by carrier
customer-service	PostgreSQL `customer_db`	Profile, addresses
catalog-sync-service	MongoDB `catalog`	Nested product JSON from PIM
store-service	PostgreSQL `store_db`	Store master, slots
order-query-service	Elasticsearch `orders-v1` index	Full-text search, fast history
notification-service	MongoDB `notifications`	Template + payload per message
audit-service	MongoDB `audit` + S3 archive	Append-only, varied shape
file-exchange-service	PostgreSQL `file_db`	Job runs, file checksums
reporting-service	PostgreSQL read replicas	No writes
chatbot-service	PostgreSQL + PGVector `chatbot_db`	SOP embeddings, chat history
api-gateway, BFFs, config, discovery, tax	None (Redis for gateway rate limits)	Stateless

4.2 Main tables and entities

order_db

- `orders` (order_id UUID PK, order_number, customer_id, channel, status, currency, subtotal, tax_total, shipping_total, grand_total, sap_order_number, created_at, updated_at, version)
- `order_lines` (line_id PK, order_id FK → orders, sku, quantity, unit_price, discount, tax, line_status, fulfillment_type)
- `order_addresses` (id PK, order_id FK, type SHIP/BILL, name, line1, city, state, zip)
- `order_status_history` (id PK, order_id FK, from_status, to_status, reason, changed_by, changed_at)
- `outbox_event` (id UUID PK, aggregate_type, aggregate_id, event_type, payload JSONB, created_at) — read by Debezium

- Relationships: orders 1–N order_lines, 1–N addresses, 1–N status history.

saga_db

- `saga_instance` (saga_id PK, order_id UNIQUE, saga_type, current_step, status, payload JSONB, started_at, updated_at)
- `saga_step_log` (id PK, saga_id FK, step, outcome, message, created_at)

inventory_db

- `location` (location_id PK, type DC/STORE, name, zip)
- `stock` (sku + location_id composite PK, on_hand, reserved, safety_stock, version)
- `reservation` (reservation_id PK, order_id, status ACTIVE/RELEASED/CONSUMED, expires_at)
- `reservation_line` (id PK, reservation_id FK, sku, location_id, quantity)

payment_db

- `payment` (payment_id PK, order_id, gateway STRIPE/PAYPAL, gateway_ref, method_token, card_last4, amount_authorized, amount_captured, amount_refunded, currency, status)
- `payment_transaction` (txn_id PK, payment_id FK, type AUTH/CAPTURE/VOID/REFUND, amount, gateway_txn_id, idempotency_key UNIQUE, status, raw_response_s3_key, created_at)
- `refund` (refund_id PK, payment_id FK, return_id, amount, reason, approved_by, status)
- `webhook_event` (event_id PK = gateway event id, type, processed_at) — stops double processing
- `reconciliation_run` (run_id PK, run_date, matched, mismatched, status)

fulfillment_db / shipping_db

- `fulfillment` (fulfillment_id PK, order_id, node_id, type SHIP/BOPIS/SFS, status)
- `fulfillment_line` (id PK, fulfillment_id FK, order_line_id, quantity)
- `shipment` (shipment_id PK, fulfillment_id, carrier, service_level, tracking_number, label_s3_key, status, shipped_at, delivered_at)
- `tracking_event` (id PK, shipment_id FK, event_code, location, event_time)

returns_db

- `return_request` (return_id PK, order_id, customer_id, status, reason, channel, flagged, created_at)
- `return_line` (id PK, return_id FK, order_line_id, quantity, condition, refund_amount)

invoice_db

- `invoice` (invoice_id PK, invoice_number UNIQUE, order_id, shipment_id, type INVOICE/CREDIT_MEMO, net, tax, total, sap_billing_doc, status)
- `invoice_line` (id PK, invoice_id FK, sku, quantity, amount, tax)

integration_db (integration-sap)

- `integration_message` (message_id PK, source_event_id UNIQUE, target SAP, idoc_type, business_key, payload JSONB, status SENT/ACKED/FAILED/DLQ, attempts, last_error, sap_idoc_number, created_at, updated_at)

customer_db: `customer` (customer_id PK = Keycloak sub, email, phone, name, tier), `address` (id PK, customer_id FK, default_flag, fields).

MongoDB **`catalog.products`**: `{ _id: sku, title, brand, categoryPath[], attributes{}, images[], returnable, shippable, taxCode, pimVersion, updatedAt }`.

Elasticsearch **`orders-v1`**: one document per order, denormalised: orderId, orderNumber, customer {id, email, phone}, status, channel, totals, lines[] (sku, title, qty), shipments[] (tracking, status), createdAt. Keyword fields for exact filters, text field for name search.

Redis keys: see section 8.

5. Integrations

All external systems are reached only through four Camel-based services: integration-sap, integration-logistics, payment-gateway-adapter and file-exchange-service. Core services never call an outside system directly.

5.1 Camel routes

Route id	Service	Source	Transformation	Destination
sap-order-out	integration-sap	Kafka `oms.order.confirmed`	Idempotency check (Redis) → OrderEvent → ORDERS05 IDoc (bean mapper)	SAP SD via `sap-idoc-destination`
sap-invoice-out	integration-sap	Kafka `oms.invoice.created`	→ INVOIC02 IDoc, invoice type	SAP SD via `sap-idoc-destination`
sap-credit-memo-out	integration-sap	Kafka `oms.payment.refunded`	→ INVOIC02 credit memo	SAP SD
sap-status-in	integration-sap	SAP `sap-idoc-server` (ALEAUD / status IDocs)	Parse status 53 / 51, match by business key	Kafka `oms.sap.order-posted`, `oms.sap.invoice-posted`, `oms.sap.failed`
sap-retry	integration-sap	REST `POST .../messages/{id}/retry`	Load stored payload from `integration_message`	Re-enters sap-order-out or sap-invoice-out
carrier-rate	integration-logistics	Kafka `oms.shipping.rate-requested`	→ carrier rate JSON (per-carrier mapper)	Carrier REST (HTTP component) → Kafka reply

Route id	Service	Source	Transformation	Destination
carrier-label	integration-logistics	Kafka `oms.shipping.label-requested`	→ carrier shipment JSON; label PDF to S3	Carrier REST → Kafka `oms.shipping.label-created`
carrier-tracking-in	integration-logistics	REST webhook `/webhooks/carriers/{carrier}`	Verify HMAC, normalise event codes	Kafka `oms.shipment.tracking-updated`
wms-ship-confirm-in	integration-logistics	JMS queue `WMS.SHIP.CONFIRM` (ActiveMQ)	XML → ShipmentConfirmed	Kafka `oms.shipment.shipped`
wms-release-out	integration-logistics	Kafka `oms.fulfillment.released`	→ WMS pick-release XML	JMS queue `WMS.PICK.RELEASE`
return-received-in	integration-logistics	JMS `WMS.RETURN.RECEIPT`	XML → ReturnReceived	Kafka `oms.return.received`
stripe-payment	payment-gateway-adapter	Kafka `oms.payment.gateway-cmd` (gateway=STRIPE)	→ Stripe PaymentIntent request + idempotency key	Stripe REST → Kafka reply
paypal-payment	payment-gateway-adapter	Same, gateway=PAYPAL	→ PayPal Orders API	PayPal REST → Kafka reply
gateway-webhook-in	payment-gateway-adapter	REST `/webhooks/stripe`, `/webhooks/paypal`	Verify signature, de-duplicate by event id	Kafka `oms.payment.gateway-event`
settlement-in	file-exchange-service	SFTP `/inbound/settlement/*.csv` (Stripe, PayPal daily)	CSV → records (Bindy), checksum	Kafka `oms.payment.settlement-received` + S3 archive
inventory-feed-in	file-exchange-service	SFTP `/inbound/inventory/*.csv` every 15 min	CSV → StockSnapshot	Kafka `oms.inventory.snapshot`
carrier-manifest-out	file-exchange-service	Timer 18:00 daily	Shipments of the day → manifest CSV	Carrier SFTP `/outbound/manifest/`
sap-recon-out	file-exchange-service	Timer 01:00 daily	Day's posted invoices → summary file	SAP SFTP `/outbound/recon/`

5.2 Error handling in every route

- `onException(SocketTimeoutException, HttpOperationFailedException 5xx, JCoException)` → 3 retries, backoff 2s ×2, then DLQ topic `oms.<target>.dlq`.
- `onException(ValidationException, MappingException)` → no retry, straight to DLQ with reason.
- Every message is logged in `integration_message` with status, attempts and last error.
- Resilience4j circuit breaker wraps each outbound endpoint (section 6).
- Kafka offsets are committed only after the route finishes (manual commit).

5.3 External systems

System	Protocol	Direction	Used for
SAP S/4HANA SD	IDoc over RFC (JCo)	Both	Sales orders, invoices, credit memos, status
Stripe	REST + webhooks	Both	Card payments, refunds
PayPal	REST + webhooks	Both	Wallet payments, refunds
Carriers (FedEx / UPS style)	REST + webhooks	Both	Rates, labels, tracking
Warehouse Management System	JMS (ActiveMQ Artemis)	Both	Pick release, ship confirm, return receipt
PIM (catalog)	Kafka topic from client	In	Product data
Tax provider	REST	Out	Sales tax calculation
Email / SMS providers	REST	Out	Customer notifications
SFTP servers (bank, carrier, SAP, WMS)	SFTP	Both	Batch files
Store POS	Kafka (client topic)	In	Store sales, in-store returns, stock moves
Amazon Bedrock	REST (Spring AI)	Out	Chatbot model

For a personal build, replace real SAP with a mock IDoc endpoint (a small Spring Boot app that accepts JSON shaped like ORDERS05), use Stripe test mode, and run an SFTP container (atmoz/sftp) and ActiveMQ Artemis in Docker.

6. Spring Cloud setup

The gateway routes only to the three BFFs and a few public APIs; services find each other through Eureka, read config from Config Server, and wrap every remote call in Resilience4j.

Note for your build: on Kubernetes, many teams drop Eureka and Config Server in favour of Kubernetes Services and ConfigMaps. Keep both here because you want to learn them; run them locally with Docker Compose first.

6.1 API Gateway routes (Spring Cloud Gateway)

Route id	Path predicate	Target	Filters
mobile	/api/v1/mobile/**	lb://mobile-bff	JWT, RequestRateLimiter (Redis, 50 req/s per user), CircuitBreaker → /fallback/mobile
web	/api/v1/web/**	lb://web-bff	JWT, rate limit, circuit breaker
admin	/api/v1/admin/**	lb://admin-bff	JWT + role check ADMIN_PORTAL, no rate limit for agents
returns	/api/v1/returns/**	lb://returns-service	JWT
stores	/api/v1/stores/**	lb://store-service	Public (no JWT), cache headers
chat	/api/v1/chat/**	lb://chatbot-service	JWT, role SUPPORT_AGENT, 10 req/min per user
webhooks-pay	/webhooks/stripe, /webhooks/paypal	lb://payment-gateway-adapter	No JWT, signature checked in service, IP allow-list
webhooks-carrier	/webhooks/carriers/**	lb://integration-logistics	No JWT, HMAC in service

Global filters: add `X-Correlation-Id` if missing, strip internal headers, log request/response time, block any `/internal/**` path.

6.2 Eureka (discovery-service)

- Standalone server, `register-with-eureka: false`; two replicas in prod, peered.
- Every service has `spring-cloud-starter-netflix-eureka-client` and registers with its `spring.application.name`.
- Calls use `lb://service-name` (gateway) or `@LoadBalanced WebClient` / OpenFeign `@FeignClient(name = "inventory-service")`.
- Heartbeat 10s, eviction 30s, so dead pods drop out quickly.

6.3 Config Server

- Backed by a Git repo `oms-config` with files like `application.yml` (shared), `order-service.yml`, `order-service-qa.yml`, `order-service-prod.yml`.
- Services use `spring.config.import: optional:configserver:http://config-server:8888` and the active profile (dev / qa / prod).
- Secrets are never in Git: values like `${STRIPE_API_KEY}` come from Kubernetes Secrets (prod: AWS Secrets Manager via External Secrets).
- `@RefreshScope` beans + `/actuator/refresh` (or Spring Cloud Bus on Kafka) to change feature flags without redeploy.

6.4 Resilience4j

Where	Pattern	Settings	Fallback
api-gateway → BFFs	Circuit breaker + timeout	50% failure over 20 calls, open 30s, timeout 3s	Friendly 503 JSON
BFFs → order-query, order, cart	Circuit breaker, timeout 2s, bulkhead 50	Same thresholds	Partial response (e.g. order without tracking block)
cart → pricing, inventory	Retry 2 + circuit breaker	Retry on 5xx and timeout only	Inventory: allow checkout, saga re-checks
pricing → tax-service → tax provider	Circuit breaker + retry 3	Open 60s	Cached tax rate by ZIP from Redis, flag order for re-tax
shipping → carrier (via Camel)	Circuit breaker per carrier	50% over 20, open 30s	Rates: cached rates from Redis; labels: wait in Kafka, retry later
integration-sap → SAP	Circuit breaker + Camel retries	Slow-call threshold 5s	None needed: message stays in Kafka, consumed after close
payment-gateway-adapter → Stripe/PayPal	Circuit breaker + time limiter 10s	No automatic retry on authorize without same idempotency key	Payment marked PENDING_RETRY, saga waits
chatbot → Bedrock	Rate limiter + timeout 20s	—	"Assistant is busy, try again"

Circuit states are exported through Actuator (`resilience4j.circuitbreaker.state`) to Prometheus, with a Grafana alert when any circuit stays OPEN more than 2 minutes.

7. Security

Keycloak issues short-lived JWTs; the gateway and every service validate them locally with Keycloak's public keys, and roles in the token decide what each caller can do.

7.1 Keycloak setup

- Realm `oms`, two user groups: customers (self-registration) and employees (federated from the client's corporate directory via LDAP/SAML).
- Clients: `mobile-app` (public, PKCE), `web-app` (public, PKCE), `admin-portal` (public, PKCE), one confidential client per backend service that calls others (client credentials).
- Access token 5 minutes, refresh token 30 minutes (customers) / 8 hours sliding (agents).

7.2 Login flow (customer or agent)

1. User opens the app; the app redirects to Keycloak login (Authorization Code flow with PKCE).
2. User signs in; Keycloak redirects back with an authorization code.
3. App exchanges code + code_verifier for access token (JWT) + refresh token.
4. App sends `Authorization: Bearer <JWT>` on every API call.
5. api-gateway (OAuth2 resource server) checks signature using Keycloak's JWKS endpoint (keys cached), plus `exp`, `iss`, `aud`.
6. Gateway forwards the token; each service validates it again (zero trust) and maps `realm_access.roles` to Spring authorities with a custom `JwtAuthenticationConverter`.
7. Before expiry the app uses the refresh token silently; logout calls Keycloak end-session.

Service-to-service: client credentials flow. The service gets its own token (cached until 30s before expiry by `OAuth2AuthorizedClientManager`) and calls with role `SERVICE`. Client secrets live in Kubernetes Secrets.

Webhooks: no JWT; verified by gateway signature (Stripe `Stripe-Signature`, PayPal verify API, carrier HMAC) plus IP allow-list.

7.3 Roles

Role	Who	Main rights
CUSTOMER	Shoppers	Own cart, own orders, own returns, own profile
SUPPORT_AGENT	Call-centre agents	Search all orders, view details, cancel unshipped orders, create returns, refunds up to $200, use chatbot
SUPPORT_SUPERVISOR	Team leads	Everything an agent can + refunds above $200 + approve flagged returns
OPS_ADMIN	Operations team	Integration screens: view and retry failed SAP / carrier messages, run SFTP jobs
FINANCE	Finance team	Payment history, reconciliation reports, daily sales reports
SERVICE	Backend services	`/internal/**` APIs only

7.4 Endpoint access by role

Endpoint group	CUSTOMER	SUPPORT_AGENT	SUPPORT_SUPERVISOR	OPS_ADMIN	FINANCE	SERVICE
Carts, checkout	Own	—	—	—	—	—
GET orders/{id}	Own	All	All	All	All	All
POST orders/{id}/cancel	Own, before ship	Yes	Yes	—	—	Yes
Order search (admin)	—	Yes	Yes	Yes	Yes	—
POST returns	Own	Yes	Yes	—	—	—
POST returns/{id}/approve	—	—	Yes	—	—	—
POST payments/{id}/refunds	—	⩽ $200	Any	—	—	Yes (auto)
GET payments, reconciliation	—	View	View	—	Yes	—
Integration messages, retry	—	View	View	Yes	—	—
File jobs run	—	—	—	Yes	—	—
Reports	—	—	Yes	Yes	Yes	—
POST chat	—	Yes	Yes	Yes	—	—
/internal/**	—	—	—	—	—	Yes

Rules are enforced with `@PreAuthorize`, for example `@PreAuthorize("hasRole('CUSTOMER') and @orderAccess.isOwner(#orderId, authentication)")`. Refund limits are checked in payment-service code, not only on the UI.

7.5 Other controls

- No card data stored or logged: only gateway tokens and last 4 digits (keeps PCI scope small).
- PII (email, phone, address) masked in logs by a Logback converter.
- TLS everywhere; inside the cluster, Kubernetes NetworkPolicies allow only expected service-to-service traffic.
- OWASP dependency check and container image scan in the pipeline.
- Every agent action (refund, cancel, retry) written to audit-service with user id and reason.

8. Redis caching

Redis does three jobs: read caching with short TTLs, idempotency keys that stop duplicate processing, and short-lived state like carts and rate limits. Nothing in Redis is the only copy of money or order data.

Key pattern	Service	What	TTL	Invalidation
`product:{sku}`	pricing, catalog-sync, BFFs	Product snapshot (title, image, tax code)	1 hour	Evicted on PIM update event
`price:{sku}`	pricing-service	Base price + active promos	10 min	Evicted on price change event
`promo:active`	pricing-service	List of live promotions	5 min	Evicted on promo publish
`tax:{zip}:{taxCode}`	tax-service	Tax rate from provider	24 hours	Time only; also fallback when provider is down
`atp:{sku}:{zip}`	inventory-service	Available-to-promise for a ZIP	30 s	Evicted on `inventory.changed`

Key pattern	Service	What	TTL	Invalidation
`store:{storeId}`	store-service	Store details, hours	6 hours	Evicted on store update
`slots:{storeId}:{date}`	store-service	BOPIS pickup slots	1 min	Evicted on booking
`rates:{carrier}:{fromZip}:{toZip}:{weightBand}`	shipping-service	Carrier rates	4 hours	Time only; also circuit-breaker fallback
`cart:{cartId}`	cart-service	Active cart	7 days (sliding)	Deleted at checkout
`customer:{id}:profile`	customer-service, BFFs	Profile summary	15 min	Evicted on profile update
`order:{orderId}:summary`	mobile-bff, web-bff	Order details response	60 s	Evicted on any order event
`idem:http:{key}`	order-service	Idempotency-Key result of POST orders	24 hours	Time only
`sap:order:{orderId}` / `sap:invoice:{invoiceId}`	integration-sap	Already sent to SAP	7 days	Time only
`pay:{idempotencyKey}`	payment-gateway-adapter	Gateway call result	24 hours	Time only
`evt:{consumer}:{eventId}`	All Kafka consumers	Event already processed	3 days	Time only
`rl:{userId}:{route}`	api-gateway	Rate-limiter tokens	Seconds	Automatic
`kc:token:{client}`	Services using client credentials	Cached service token	Token expiry – 30 s	Automatic
`chat:{sessionId}`	chatbot-service	Last 10 chat turns	30 min	Automatic

How it is coded. Read caches use Spring Cache (`@Cacheable(cacheNames = "product", key = "#sku")`) with a `RedisCacheManager` that sets a TTL per cache name in config. Idempotency uses `SET key value NX EX ttl` through `StringRedisTemplate`, so the check-and-set is one atomic step. Cache-aside pattern everywhere; a cache miss always falls back to the owning service or database.

Prod setup. ElastiCache Redis cluster mode, 3 shards with 1 replica each; `allkeys-lru` eviction for cache keys, idempotency keys in a separate logical cluster with `noeviction` so they are never silently dropped.

9. Angular frontend (admin portal)

The Angular app is the internal admin portal for agents, supervisors, operations and finance; it calls only admin-bff (plus chatbot and returns through the gateway). The customer storefront and the React Native app are outside this build.

9.1 Modules and structure

Module (lazy-loaded)	Purpose	Route guard
CoreModule	HTTP interceptors (JWT, correlation id, error), auth service (keycloak-angular), layout, global error toast	—
SharedModule	Reusable components: data table with server paging, status badge, money pipe, date pipe, confirm dialog	—
OrdersModule	Order search, order details, timeline, cancel	SUPPORT_AGENT+
ReturnsModule	Return queue, return details, approve/reject	SUPPORT_AGENT+, approve = SUPERVISOR
PaymentsModule	Payment history, refund dialog	SUPPORT_AGENT+
IntegrationsModule	Failed SAP and carrier messages, retry, SFTP job runs	OPS_ADMIN
ReportsModule	Daily sales, refunds, SLA dashboards	FINANCE, SUPERVISOR
ChatModule	Chatbot side panel	SUPPORT_AGENT+

Stack: Angular 17, Angular Material, RxJS, NgRx (only for orders search state and user session), keycloak-angular for login, ngx-charts for reports. Folder layout `src/app/core`, `shared`, `features/<module>/{components,pages,services,models}`.

9.2 Main screens and the APIs they call

Screen	What the user does	API calls (via gateway → admin-bff)
Login	Keycloak redirect	Keycloak only
Dashboard	Today's orders, failed integrations count, open flagged returns	GET /admin/dashboard/summary

Screen	What the user does	API calls (via gateway → admin-bff)
Order search	Filter by order no, email, phone, status, date range; cursor paging	GET /admin/orders/search?query=&status=&from=&to=&cursor=
Order details	Header, lines, addresses, payment, shipments	GET /admin/orders/{id}, GET /admin/orders/{id}/payments, GET /admin/orders/{id}/shipments
Order timeline	Every event from placed to delivered, incl. SAP status	GET /admin/orders/{id}/timeline
Cancel order	Reason dropdown, confirm	POST /admin/orders/{id}/cancel
Create return	Select lines, quantity, reason, condition	POST /admin/returns
Return queue	Flagged returns, approve / reject	GET /admin/returns?status=REVIEW, POST /admin/returns/{id}/approve
Refund dialog	Amount, reason, note; above $200 needs supervisor	POST /admin/payments/{paymentId}/refunds
Failed integrations	List DLQ messages for SAP and carriers, view payload, retry, bulk retry	GET /admin/integrations/messages?target=SAP&status=FAILED, POST /admin/integrations/messages/{id}/retry
File jobs	See SFTP job runs, re-run a job	GET /admin/files/jobs, POST /admin/files/jobs/{name}/run
Reports	Charts and CSV export	GET /admin/reports/daily-sales, /refunds, /sla
Chat panel	Ask questions about an order or process	POST /api/v1/chat

9.3 Reactive forms

- **Order search form:** `FormGroup` with orderNumber, email, phone, status (multi-select), dateFrom, dateTo. Custom validator: at least one field filled; date range max 90 days. `valueChanges` with `debounceTime(400)` + `distinctUntilChanged` + `switchMap` to call search.
- **Refund form:** amount (required, min 0.01, max = captured minus refunded, async validator that checks current balance), reason (required), note (max 500). If amount > 200 and user lacks SUPERVISOR, the form shows "needs approval" and submits as a

request instead.
- **Return form:** `FormArray` of lines, each with quantity (max = shipped quantity) and condition; at least one line selected.
- **Cancel form:** reason dropdown + note; disabled if any line already shipped.
- **Retry form (bulk):** selected message ids + optional corrected field values for data errors.

9.4 Cross-cutting

- `AuthInterceptor` adds the Bearer token; `ErrorInterceptor` maps the standard error body (section 14) to a toast and logs the traceId so agents can quote it.
- Role-based guards (`canMatch`) hide whole modules; `*hasRole` directive hides buttons.
- Server-side pagination everywhere; no screen loads more than 50 rows at once.
- Unit tests with Jasmine/Karma for services and pipes; Cypress for 5 critical journeys (search, refund, return approve, retry, cancel).

10. Spring AI chatbot

The chatbot answers support agents' questions about a specific order and about support procedures, by calling existing APIs (tool calling) and searching SOP documents (RAG); it is read-only and never changes data.

What agents ask, for example

- "Why is order 10045 not invoiced yet?" → looks up the order timeline and the SAP message status.
- "Where is the package for order 10045?" → shipment and tracking events.
- "Was the customer refunded?" → payment history.
- "How do I handle a damaged-item claim over $500?" → SOP search.

How it is built (chatbot-service)

1. Angular chat panel calls `POST /api/v1/chat` with `{sessionId, message, orderId?}` and the agent's JWT.
2. chatbot-service uses Spring AI `ChatClient` with Amazon Bedrock as the model (any Spring AI-supported model works for your build, including a local Ollama model).
3. A system prompt sets the rules: answer only from tool results and documents, say "I don't know" otherwise, never reveal card or full personal data.
4. Tools are Spring beans annotated with `@Tool`, each calling an existing service with the **agent's own JWT** (token relay), so the bot sees only what the agent may see:

- `getOrderSummary(orderId)` → order-query-service
- `getOrderTimeline(orderId)` → admin-bff
- `getShipmentTracking(orderId)` → shipping-service
- `getPaymentHistory(orderId)` → payment-service
- `getIntegrationStatus(orderId)` → integration-sap / integration-logistics

5. RAG: SOP and policy documents (PDF/Markdown) are split into chunks, embedded, and stored in PGVector (`VectorStore`). A `QuestionAnswerAdvisor` adds the top 4 matching chunks to the prompt.
6. Chat memory: last 10 turns per session in Redis (`chat:{sessionId}`), via a chat-memory advisor.
7. Response streams back to the UI (`Flux<String>`, server-sent events), with source links to the SOP chunks used.

Safety and limits

- Only read tools exist; no tool can refund, cancel or retry.
- PII masking on tool outputs before they go to the model.
- Rate limit 10 requests per minute per agent; timeout 20s.
- Every question and tool call logged to audit-service.
- Admin job re-indexes SOP documents when they change (`POST /internal/v1/chat/reindex`).

11. DevOps

Jenkins builds and tests every service into a Docker image; Argo CD deploys it to Kubernetes from a GitOps repo; Prometheus and Grafana watch it through Actuator metrics.

11.1 Jenkins pipeline (one shared `Jenkinsfile` library for all services)

1. **Checkout** — feature branch or develop / release branch.
2. **Build** — `mvn -B clean verify -DskipITs` (compile + unit tests).
3. **Unit test report + coverage** — JUnit and JaCoCo; build fails under 80% line coverage on service and mapper packages.
4. **Integration tests** — `mvn verify -Pit` with Testcontainers (PostgreSQL, Kafka, Redis).
5. **Static analysis** — SonarQube scan, quality gate must pass.
6. **Security scan** — OWASP dependency check; Trivy scan of the image after build.
7. **Docker build** — multi-stage Dockerfile, tag `<service>:<gitSha>`.

8. **Push** — to Amazon ECR (Docker Hub or a local registry for your build).
9. **Update GitOps repo** — Jenkins bumps the image tag in `oms-deploy/<service>/values-dev.yaml` and commits.
10. **Deploy dev** — Argo CD auto-syncs dev.
11. **API tests** — Postman collection via Newman against dev; failure stops promotion.
12. **Promote to QA** — on release branch; Argo CD auto-sync to QA.
13. **Prod** — manual approval step in Jenkins after QA sign-off, then tag bump in `values-prod.yaml`; Argo CD syncs with a manual sync window.

11.2 Docker

- Multi-stage: `maven:3.9-eclipse-temurin-17` to build, `eclipse-temurin:17-jre-alpine` to run (Spring Boot layered jar for faster pulls).
- Non-root user, `JAVA_TOOL_OPTIONS=-XX:MaxRAMPercentage=75`.
- Angular: build with Node, serve static files with nginx.
- For local work: one `docker-compose.yml` with PostgreSQL, MongoDB, Redis, Kafka (KRaft), Kafka Connect + Debezium, Elasticsearch, Keycloak, ActiveMQ Artemis, SFTP, Prometheus, Grafana, Jaeger.

11.3 Kubernetes setup

Object	How it is used
Namespaces	`oms-edge` (gateway, BFFs), `oms-core` (domain services), `oms-integration` (Camel services), `oms-platform` (config, discovery, Keycloak), `oms-observability` (Prometheus, Grafana, Jaeger, ELK)
Deployments	One per service; 3 replicas min in prod for core, 2 for supporting; rolling update `maxUnavailable: 0`, `maxSurge: 1`
Services	`ClusterIP` for every service; only an `Ingress` (AWS ALB) in front of api-gateway
ConfigMaps	Non-secret settings (profile name, Config Server URL, log level, feature flags)
Secrets	DB passwords, Keycloak client secrets, Stripe/PayPal keys, SAP JCo credentials; in prod synced from AWS Secrets Manager by External Secrets Operator

Object	How it is used
Probes	Liveness `/actuator/health/liveness`, readiness `/actuator/health/readiness`
HPA	CPU 70% for APIs; Kafka consumer lag (via KEDA) for integration and saga services; min 3, max 20
PodDisruptionBudget	`minAvailable: 2` for core services
Resources	Requests 500m CPU / 768Mi, limits 1 CPU / 1Gi (typical service)
Helm	One shared chart `oms-service` with per-service values files
NetworkPolicies	Only gateway can reach BFFs; only BFFs and services in `oms-core` can call core services

11.4 Environments

Env	Cluster	Data	Deploy	Purpose
local	Docker Compose	Seed data	Manual	Your laptop
dev	Shared EKS, namespace per team	Synthetic	Auto on merge to develop	Developer testing
QA	Separate EKS	Masked copy of prod-like data	Auto from release branch	QA, regression, load tests
prod	Separate EKS, 3 availability zones	Real	Manual approval	Customers

Infra (EKS, RDS, MSK, ElastiCache, S3) is created by Terraform modules, one state per environment.

11.5 Monitoring

- **Actuator** on every service: `health`, `info`, `prometheus`, `metrics` exposed on a separate management port (8081), not through the gateway.
- **Micrometer** custom metrics: `orders_created_total`, `payment_auth_failures_total`, `sap_idoc_sent_total{status}`, `saga_duration_seconds`, plus Kafka consumer lag.
- **Prometheus** scrapes pods through ServiceMonitors (kube-prometheus-stack).

- **Grafana** dashboards: platform overview, per-service RED (rate, errors, duration), Kafka lag, circuit breakers, SAP integration, JVM.
- **Alerts** (Alertmanager → Teams / PagerDuty): error rate > 2% for 5 min, p95 latency > 800 ms, consumer lag > 5,000, DLQ message count > 0, circuit OPEN > 2 min, pod restarts > 3 in 10 min.
- **Logs**: JSON logs → Filebeat → Logstash → Elasticsearch → Kibana, searchable by `traceId` and `orderId`.
- **Tracing**: Micrometer Tracing + OpenTelemetry exporter → Jaeger; trace id travels in HTTP headers and Kafka headers.

12. Testing

Unit tests cover business rules in isolation with JUnit 5 and Mockito; integration tests run each service against real PostgreSQL, Kafka, Redis and mocked external systems in Testcontainers.

12.1 Unit tests (JUnit 5 + Mockito + AssertJ)

Area	Examples
Service layer	OrderService: create order writes outbox event; cancel rejected after shipment
Pricing rules	Promo stacking, coupon expiry, rounding to 2 decimals, tax added per line
Saga logic	Orchestrator picks the right next command; failure at payment triggers release + cancel
Inventory	Reserve fails when on_hand − reserved < requested; release restores quantity
Payment	Refund amount cannot exceed captured − refunded; agent limit $200 enforced
Fraud rules	Each rule tested with edge values (e.g. 3rd return in 30 days flags)
Mappers	MapStruct / Camel bean mappers: OrderEvent → ORDERS05 fields, Invoice → INVOIC02 totals
Idempotency	Second call with same key returns stored result, gateway mock called once

Area	Examples
Exception handler	Each custom exception maps to the right HTTP status and error code
Controllers	`@WebMvcTest` with MockMvc: validation errors return 400 with field list; role checks with `@WithMockUser`

Mocks: repositories, Feign clients, KafkaTemplate, RedisTemplate, Clock (for time rules).

12.2 Integration tests

Area	Tool	What is checked
Repositories	`@DataJpaTest` + Testcontainers PostgreSQL	Queries, indexes used, optimistic locking
Outbox → Kafka	Testcontainers Kafka + Debezium (or outbox poller in tests)	Event published once per order
Kafka consumers	`@SpringBootTest` + Testcontainers Kafka	Consume, process, duplicate event ignored, failed message goes to DLQ
Camel routes	`CamelSpringBootTest`, `AdviceWith`, `MockEndpoint`, WireMock for carriers / Stripe	Mapping, retry count, DLQ on validation error
SAP route	Mock SAP endpoint (replace `sap-idoc-destination` with mock)	IDoc fields, status handling
SFTP routes	Testcontainers SFTP (atmoz/sftp)	File picked, parsed, moved to `/processed`
Security	Testcontainers Keycloak	Real token accepted, wrong role gets 403
Redis	Testcontainers Redis	TTLs, SETNX idempotency
Contracts	Spring Cloud Contract between BFFs and core services	API changes do not break consumers
End to end (QA env)	Postman + Newman, Cypress for UI	Order → payment → ship → invoice → refund happy and failure paths

Area	Tool	What is checked
Load (QA env)	Gatling	3,000 orders/min for 30 min with SAP mock adding 3–4 s delay

13. Performance work

The main latency wins came from fixing queries and N+1 loading, adding the right indexes, moving reads to Elasticsearch and Redis, and tuning connection pools and Kafka consumers.

Problem	Change	Effect
Order details API loaded lines and addresses with N+1 queries	`@EntityGraph(attributePaths = {"lines", "addresses"})` / `JOIN FETCH`; DTO projection instead of full entities	p95 420 ms → 90 ms
Order history used `OFFSET` paging on PostgreSQL	Moved to order-query-service on Elasticsearch with `search_after` cursor	Under 100 ms even for customers with thousands of orders
Slow lookups by customer and status	Composite index `orders(customer_id, created_at DESC)`, partial index `orders(status) WHERE status IN ('PENDING','CONFIRMED')`, index on `payment_transaction(idempotency_key)`	Seq scans removed (checked with `EXPLAIN ANALYZE`)
Bulk status updates one row at a time	`hibernate.jdbc.batch_size=50`, `order_inserts`, `order_updates`, sequence `allocationSize=50`	Batch job 6x faster
Open Session in View held DB connections during JSON rendering	`spring.jpa.open-in-view=false`	Fewer pool timeouts at peak
Hikari pool exhaustion on sale days	Pool sized per pod (max 20), `connectionTimeout` 3 s, read replica for reporting	No pool-wait errors in last sale
Price and product lookups on every cart call	Redis cache with TTLs (section 8)	Cart API p95 300 ms → 70 ms

Problem	Change	Effect
BFF called 4 services one after another	Parallel calls with `WebClient` + `Mono.zip`	Order details BFF 600 ms → 220 ms
Large JSON responses	Only needed fields in DTOs; gzip compression at gateway	Payload size down about 60%
Kafka consumer rebalances when SAP was slow	`max.poll.records` 500 → 50, longer `max.poll.interval.ms`, consumers 3 → 6 pods	Lag cleared; no rebalance loops
Inventory hot rows (popular SKUs)	Short transactions, `SELECT ... FOR UPDATE SKIP LOCKED` per location, optimistic `@Version` elsewhere	Fewer lock waits

The numbers above are targets to measure in your own build with Gatling and Grafana; treat them as the kind of result to look for, not fixed facts.

14. Common standards (all services)

Every service is generated from one template repo, so structure, errors, logs and API responses look the same everywhere.

14.1 Package and folder structure

```
order-service/
  src/main/java/com/retail/oms/order/
    OrderServiceApplication.java
    api/            controllers, request/response DTOs
    application/    services (use cases), saga handlers
    domain/         entities, enums, domain rules
    infrastructure/
      persistence/  JPA repositories
      messaging/    Kafka producers, consumers, outbox
      client/       Feign / WebClient clients
      config/       security, kafka, redis, openapi config
    mapper/         MapStruct mappers
    exception/      custom exceptions, global handler
  src/main/resources/
    application.yml, db/migration (Flyway V1__init.sql ...)
  src/test/java/... unit/, it/
  Dockerfile, Jenkinsfile, helm values in oms-deploy repo
```

Shared code lives in small internal libraries: `oms-common-web` (error handler, response wrapper, correlation filter), `oms-common-events` (event schemas, Kafka headers), `oms-common-security` (JWT role converter).

14.2 Exception handling

- Base class `OmsException(errorCode, httpStatus, message)`; subclasses like `OrderNotFoundException` (404, ORD-404), `InvalidOrderStateException` (409, ORD-409), `InsufficientStockException` (409, INV-409), `PaymentDeclinedException` (402, PAY-402), `RefundLimitExceededException` (403, PAY-403).
- One `@RestControllerAdvice` in `oms-common-web` handles: `OmsException`, `MethodArgumentNotValidException` (400 with field errors), `AccessDeniedException` (403), `CallNotPermittedException` from Resilience4j (503), and `Exception` (500, generic message only).
- Stack traces go only to logs, never to the client.
- Kafka consumers: retryable errors use `DefaultErrorHandler` with backoff then a dead-letter topic; non-retryable exceptions are listed so they skip retries.

14.3 Logging

- SLF4J + Logback, JSON format (logstash-logback-encoder) in all non-local environments.
- MDC fields on every line: `traceId`, `spanId`, `correlationId`, `orderId` (when known), `userId`.
- Levels: INFO for business events (order created, IDoc sent), WARN for retries, ERROR only for failures needing attention.
- Never log card data, tokens, passwords or full addresses; a masking converter covers email and phone.

14.4 API response format

Success (single resource): plain DTO body with correct status (200, 201 + `Location` header).

Success (list):

```
{
  "data": [ ... ],
  "page": { "size": 20, "nextCursor": "eyJ0cyI6...", "total": 1342 }
}
```

Error (every service, every error):

```
{
  "timestamp": "2026-10-29T10:15:30Z",
  "status": 409,
  "errorCode": "ORD-409",
  "message": "Order cannot be cancelled after shipment",
  "path": "/api/v1/orders/9f3c.../cancel",
  "traceId": "4bf92f3577b34da6",
  "fieldErrors": [ { "field": "reason", "message": "must not be blank" } ]
}
```

14.5 Naming conventions

Thing	Convention	Example
Service / repo	kebab-case, `-service` suffix	`order-service`
Java package	`com.retail.oms.<service>`	`com.retail.oms.payment`
REST paths	plural nouns, kebab-case, versioned	`/api/v1/return-requests/{id}`
JSON fields	camelCase	`orderNumber`
DB tables / columns	snake_case, singular or plural kept consistent per DB	`order_lines.unit_price`
Kafka topics	`oms.<domain>.<event-in-past-tense>` or `-cmd` for commands	`oms.payment.captured`, `oms.inventory.reserve-cmd`
Events (classes)	Past tense noun	`OrderCreatedEvent`
Error codes	`<DOMAIN>-<HTTP>`	`PAY-402`
Config keys	`oms.<service>.<setting>`	`oms.payment.refund-agent-limit=200`
Git branches	`feature/OMS-1234-short-desc`, `bugfix/...`, `hotfix/...`	`feature/OMS-2210-paypal-refunds`
Flyway scripts	`V<number>__<desc>.sql`	`V7__add_refund_table.sql`

15. Team and process

A 9-person Scrum team works in two-week sprints: stories come from the client's product owner through Jira, code goes through pull requests on a GitFlow-style branch model, and releases reach production every two weeks through QA sign-off and a manual approval.

15.1 Team

Role	Count	Main job
Tech lead	1	Design, reviews of risky changes, client technical calls
Developers (full stack / backend)	5	Stories end to end, production support rotation
QA engineer	1	Test cases, Postman/Newman, Cypress, regression
DevOps engineer	1	Jenkins, Kubernetes, Helm, Argo CD, Terraform, monitoring
Scrum Master	1	Ceremonies, blockers, sprint reporting
Client side	—	Product owner (priorities), BA (requirements), SAP team, React Native and storefront teams

15.2 How work comes in

1. Product owner and BA write epics and stories in Jira (`OMS-xxxx`) with acceptance criteria and API contract notes.
2. Backlog refinement mid-sprint: team asks questions, splits stories, adds technical sub-tasks.
3. Sprint planning: stories estimated in story points (velocity about 40–45 per sprint), sprint goal agreed.
4. Definition of Ready: acceptance criteria clear, API contract agreed with consuming team, test data known.
5. Definition of Done: code merged, unit + integration tests passing, Sonar gate green, deployed to QA, QA passed, Swagger / Confluence updated.
6. Production bugs come as Jira bugs with priority P1–P4; P1 goes straight into the sprint.

15.3 Git branching

- `main` = what is in production; tagged per release (`v2026.10.2`).

- `develop` = integration branch, auto-deployed to dev.
- `feature/OMS-1234-...` branched from develop, merged back by PR.
- `release/2026.10.2` cut from develop at sprint end, deployed to QA; only fixes go in; merged to main and back to develop after release.
- `hotfix/OMS-xxxx-...` from main for urgent prod fixes, merged to main and develop.
- Commit messages start with the Jira key: `OMS-2210 add PayPal refund webhook handling`.

15.4 Code review

- Every change goes through a pull request; at least 1 approval, 2 for money flows (payment, refund, invoice) and security config; tech lead must approve those.
- Jenkins PR build must be green (tests, coverage, Sonar) before merge.
- PR template asks for: Jira link, what changed, how tested, DB migration (yes/no), config change (yes/no), rollback note.
- Review checklist: idempotency for anything consuming Kafka or calling a gateway, no secrets in code, logs without PII, errors use standard codes, tests for the failure path.

15.5 Release to production

1. Release branch deployed to QA; full regression (Newman + Cypress) and, before big sales, a Gatling load test.
2. QA sign-off and product owner demo sign-off recorded in Jira.
3. Change request raised with the client (release notes, rollback plan, DB migrations listed).
4. Release window (low-traffic US night, around 10:30 am IST); Jenkins manual approval by tech lead.
5. Image tags bumped in `values-prod.yaml`; Argo CD syncs; rolling update with readiness probes, zero downtime.
6. Smoke tests on prod (health checks, a test order on a test store with Stripe test mode, dashboards).
7. Watch Grafana for 30 minutes; any error spike → rollback in Argo CD to the previous revision.
8. Code freeze two weeks before Black Friday: only P1 fixes.

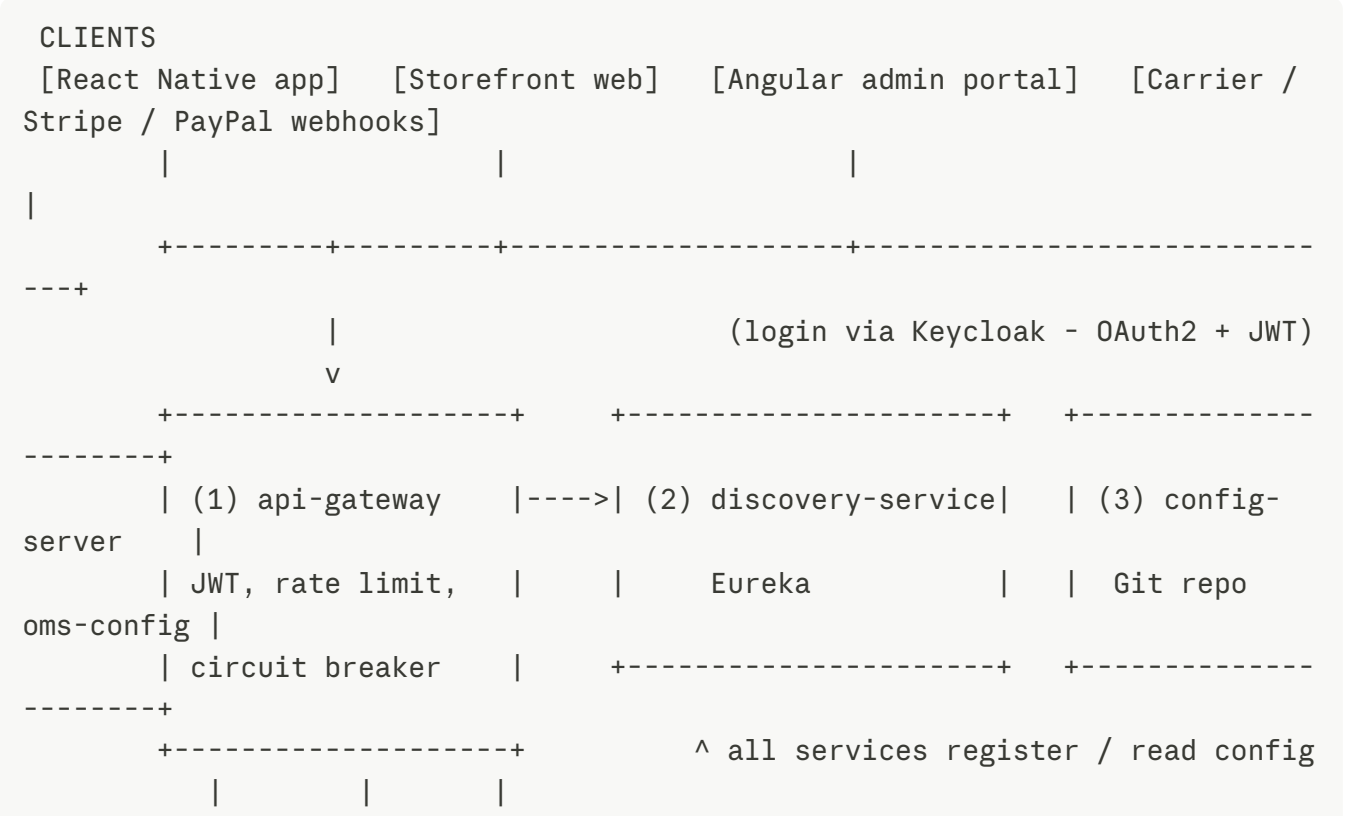
15.6 Suggested build order for your learning project

You do not need all 30 services on day one. Build in this order, each step working end to end before the next:

1. **Foundation:** config-server, discovery-service, api-gateway, Keycloak, one shared template with error handling and logging.
2. **Order core:** cart-service, order-service with outbox, pricing-service (no tax yet), order-query-service on Elasticsearch.
3. **Saga:** order-orchestrator, inventory-service, payment-service + payment-gateway-adapter with Stripe test mode.
4. **Fulfillment:** fulfillment-service, shipping-service, integration-logistics with a mock carrier (WireMock) and ActiveMQ.
5. **SAP and invoices:** invoice-service, integration-sap with a mock IDoc endpoint, DLQ and retry screen.
6. **Angular admin portal:** search, details, timeline, refund, failed integrations.
7. **Returns and fraud:** returns-service, fraud-service, refund flow.
8. **Ops:** Docker Compose → local Kubernetes (kind or minikube) → Jenkins pipeline → Prometheus/Grafana.
9. **Extras:** chatbot-service with Spring AI, file-exchange-service with SFTP, notification, audit, reporting, BFFs.

16. Architecture diagram

Read it top to bottom: clients enter through the gateway and BFFs, core services talk mainly over Kafka, and only the four integration services touch outside systems. Numbers match the service catalog in section 2.




(4) mobile-bff (5) web-bff (6) admin-bff
[Keycloak]
sync REST (reads, checkout)
(30) chatbot-service
CORE DOMAIN
Spring AI, Bedrock,
(7) cart --> (10) pricing --> (11) tax --> [Tax provider]
PGVector, tools call
order-query, shipping,
payment (read-only)
(8) order-service --outbox--> [Debezium] --> KAFKA
(9) order-orchestrator (saga) <== commands / replies ==>
(12) inventory (15) fraud (13) payment (16) fulfillment
(17) shipping (19) returns (20) invoice
(22) customer (23) catalog-sync (24) store
KAFKA (Amazon MSK)
oms.order.*, oms.payment.*, oms.inventory.*,
oms.shipment.*, oms.invoice.*, oms.sap.*, *.dlq
(25) order-query Elasticsearch
(26) notification email/SMS/push
(27) audit Mongo + S3
(29) reporting read replicas
INTEGRATION LAYER (Apache Camel)
(21) integration-sap
ORDERS05, INVOIC02,
status IDocs, Redis
idempotency, DLQ
(18) integration-logistics
REST + JMS + webhooks
settlement, stock,
manifests, SAP recon
(14) payment-gateway-adapter
Stripe, PayPal
webhooks
(28) file-exchange
SFTP jobs

```
            v                    v           v                    v
v
     [SAP S/4HANA SD]     [Carriers APIs]  [ActiveMQ <-> WMS]  [Stripe]
[PayPal]      [SFTP servers: bank,
      (IDoc / JCo)
carrier, SAP, WMS]

 DATA STORES (one per service)
 PostgreSQL: order, saga, cart, pricing, inventory, payment, fraud,
fulfillment, shipping,
             returns, invoice, integration, customer, store, file, chatbot
(+PGVector)
 MongoDB:    catalog, notifications, audit, carrier payloads
 Elasticsearch: orders-v1 (CQRS read side)      Redis: cache, idempotency,
carts, rate limits
 S3: labels, raw payloads, audit archive

 PLATFORM
 Jenkins -> ECR -> GitOps repo -> Argo CD -> Kubernetes (EKS: oms-edge, oms-
core,
 oms-integration, oms-platform, oms-observability)
 Actuator/Micrometer -> Prometheus -> Grafana + Alertmanager | Filebeat ->
ELK | OTel -> Jaeger
```